Universidade de São Paulo (USP)
Instituto de Ciências Matemáticas e de Computação (ICMC)

SCC0230 – Inteligência Artificial
Prof. Alneu de Andrade Lopes

Algoritmos de Busca em Redes KNN

Abordagem A – Projeto de Busca em Redes

Grupo:
Alec Campos Aoki (NUSP 15436800)
Daniel Jorge Manzano (NUSP 15446861)
Fernando Valentim Torres (NUSP 15452340)
Frederico Scheffel Oliveira (NUSP 15452718)
Pedro Henrique Perez Dias (NUSP 15484075)

Resumo

Este trabalho implementa e compara seis algoritmos de busca (busca em profundidade, busca em largura, Best-First, A*, Hill Climbing sem backtracking e Dijkstra) em redes KNN, grafos em que cada vértice, posicionado aleatoriamente no plano, é ligado aos seus k vizinhos mais próximos. Os algoritmos e o gerador de redes foram implementados em Python sem bibliotecas de grafos, e cada busca grava os eventos de fronteira e expansão, o que permite animar passo a passo a exploração do espaço de busca. Em redes de 2500 vértices com $k \in \{3, 7, 11\}$ e 30 pares origem–destino, A* e Dijkstra encontram sempre a mesma distância, que é a ótima, mas o A* expande de 67% a 90% menos vértices, e essa vantagem cresce com k e com a distância entre origem e destino. Com a heurística de Chebyshev, também admissível, porém menos precisa, o A* mantém a otimalidade, mas perde parte dessa vantagem. O Best-First chega a soluções próximas da ótima expandindo poucos vértices, a busca em profundidade produz caminhos de 7 a 50 vezes mais longos que o ótimo, e o Hill Climbing tem taxa de sucesso que vai de 7% com $k = 3$ a 100% com $k = 11$, o que ilustra diretamente o problema dos mínimos locais.

1 Introdução

Problemas de busca em grafos aparecem em roteamento de veículos, jogos, planejamento e redes de comunicação. Os algoritmos estudados na disciplina se dividem em dois grupos. As buscas não informadas, como a busca em largura (BFS), a busca em profundidade (DFS) e o algoritmo de Dijkstra, exploram o grafo sem nenhuma informação sobre a posição do destino. As buscas informadas, como o Best-First, o A* e o Hill Climbing, usam uma heurística $h(n)$ que estima o custo restante de um vértice n até o destino para decidir o que explorar primeiro [1]. O algoritmo de Dijkstra [3] e o A* [2] são os dois que garantem o caminho de menor custo, e a diferença entre eles está justamente no uso da heurística.

Todos esses algoritmos seguem o mesmo ciclo: retiram um vértice da *fronteira* (o conjunto de vértices já descobertos e ainda não explorados), verificam se ele é o destino, *expandem* o vértice examinando seus vizinhos e inserem na fronteira os vizinhos novos. O que distingue um algoritmo do outro é apenas o critério que decide qual vértice sai da fronteira, ou seja, a estrutura de dados usada nela. Por isso, comparar esses algoritmos sobre a mesma rede e os mesmos pares origem–destino mostra com clareza o efeito de cada critério sobre a qualidade da solução e sobre o esforço da busca.

O grupo escolheu a rede KNN porque ela tem uma geometria natural: as arestas ligam vértices próximos no plano e o peso de cada aresta é a distância euclidiana entre suas pontas. Isso torna a distância em linha reta uma heurística admissível e intuitiva, e deixa as visualizações legíveis, já que as arestas são locais e o desenho se parece com uma malha de estradas. Os objetivos do trabalho são: (i) implementar o gerador de redes e os seis algoritmos; (ii) visualizar como cada algoritmo explora o espaço de busca em uma rede pequena; (iii) medir distância, tempo e nós expandidos em redes de 2500 vértices; e (iv) comparar em detalhe o A* com o Dijkstra, incluindo o efeito da escolha da heurística.

2 Implementação

Tudo foi implementado em Python 3, usando apenas `numpy` na geração das redes, `heapq` e `collections.deque` nas estruturas de fronteira, `matplotlib` nas visualizações e `pandas` na agregação dos resultados. Nenhuma biblioteca de grafos foi usada. O código está disponível em `https://github.com/Alec-Aoki/busca-em-redes-knn/tree/envio`.

2.1 Geração da rede KNN

Seguindo o enunciado, os n vértices recebem coordenadas sorteadas uniformemente em $[0, n] \times [0, n]$, de modo que o plano cresce com o número de vértices e a densidade de pontos se mantém. Para cada vértice, são encontrados os k vizinhos mais próximos por força bruta: calcula-se a matriz $n \times n$ de distâncias euclidianas com operações vetorizadas, a diagonal recebe infinito (um vértice não pode ser vizinho de si mesmo) e a função `argpartition` separa os k menores valores de cada linha em tempo $O(n)$, sem ordenar a linha inteira. Para $n = 2500$, a geração leva menos de meio segundo, e por isso a força bruta foi preferida a estruturas como a KD-Tree, que só compensariam em redes bem maiores.

O grafo é não-direcionado e ponderado: a aresta (A, B) existe se B está entre os k mais próximos de A *ou* se A está entre os k mais próximos de B, e seu peso é a distância euclidiana entre os dois. Como a relação de vizinhança não é simétrica, o grau médio real fica acima de k: com $n = 2500$, ele é 3,73 para $k = 3$, 8,16 para $k = 7$ e 12,51 para $k = 11$. A conectividade também depende de k. Com $k = 3$, a rede tem 8 componentes conexas (a maior com 2448 vértices), enquanto com $k = 7$ e $k = 11$ ela é conexa. Por isso, todos os pares origem–destino são sorteados dentro da maior componente, o que garante a existência de um caminho.

2.2 Heurística

As buscas informadas usam $h(n)$ igual à distância euclidiana de n até o destino. Como o peso de cada aresta é a distância euclidiana entre suas pontas, a desigualdade triangular garante que nenhum caminho pelo grafo é mais curto que a linha reta, ou seja, h nunca superestima o custo real, e portanto é *admissível*. Pelo mesmo argumento, $h(u) \leq w(u, v) + h(v)$ para toda aresta (u, v), então h também é *consistente*. Com uma heurística consistente, o A* encontra o caminho ótimo e nunca precisa reexpandir um vértice [2].

No experimento iii também foi usada a distância de Chebyshev, $h_C(n) = \max(|x_n - x_d|, |y_n - y_d|)$, onde (x_d, y_d) são as coordenadas do destino. Em um triângulo retângulo com catetos $|x_n - x_d|$ e $|y_n - y_d|$, ela é o cateto maior, enquanto a distância euclidiana é a hipotenusa. Logo, $h_C(n) \leq h(n)$ para todo vértice, e h_C também é admissível e consistente. A diferença é que ela estima pior o custo restante: coincide com a euclidiana quando o destino está na mesma linha ou coluna, mas subestima a distância em cerca de 29% quando ele está na diagonal. Isso permite observar o efeito da qualidade da heurística sobre o A*, mantendo a garantia de otimalidade.

2.3 Algoritmos de busca

Os seis algoritmos têm a mesma assinatura: recebem o grafo, a origem, o destino e um indicador de gravação de eventos, e devolvem uma estrutura com o caminho, a distância, o número de nós expandidos e a lista de eventos. Cada vértice alcançado guarda o vértice de onde veio (o seu *pai*), e o caminho é reconstruído no fim, partindo do destino e seguindo os pais até a origem. A distância é sempre recalculada a partir do caminho final, somando os pesos das arestas, para que seja comparável inclusive nos algoritmos que não acumulam custo. Em todos eles, o teste de objetivo é feito quando o vértice é expandido, e não quando entra na fronteira, para que a contagem de nós expandidos seja comparável entre os algoritmos. A Tabela 1 resume as diferenças.

Tabela 1: Critério de escolha e estrutura da fronteira de cada algoritmo.

Algoritmo	Fronteira	Comportamento
DFS	pilha	Expande o vértice descoberto mais recentemente. Implementada de forma iterativa, para não estourar o limite de recursão do Python em redes grandes. Não garante caminho mínimo.
BFS	fila	Expande o vértice descoberto há mais tempo, crescendo em camadas. Minimiza o número de arestas, mas ignora os pesos.
Best-First	heap por $h(n)$	Expande o vértice que parece mais perto do destino, sem considerar o custo já percorrido. Não garante caminho mínimo.
A*	heap por $g(n) + h(n)$	Combina o custo já percorrido $g(n)$ com a estimativa restante. Ótimo com heurística admissível.
Hill Climbing	não tem	Guarda apenas o vértice atual e se move para o vizinho de menor h, se ele for estritamente melhor que o atual. Sem backtracking: em um mínimo local, para e reporta falha.
Dijkstra	heap por $g(n)$	Expande o vértice mais perto da origem. Ótimo, mas sem informação sobre a direção do destino.

Dois detalhes de implementação merecem destaque. No A* e no Dijkstra, quando um caminho melhor até um vértice é encontrado, sua prioridade precisa diminuir, mas o `heapq` do Python não oferece essa operação. A solução adotada insere uma nova entrada com a prioridade atualizada e descarta as entradas antigas quando elas saem do heap, reconhecendo-as por já terem sido expandidas ou por terem custo maior que o melhor conhecido. Na DFS, um mesmo vértice pode ser empilhado várias vezes por pais diferentes, e por isso a pilha guarda pares (vértice, pai) e o pai é definido no momento em que o vértice é desempilhado. Os códigos do A* e do Dijkstra são idênticos, exceto pela parcela $h(n)$ somada à prioridade, o que deixa explícito que o A* é um Dijkstra guiado por uma estimativa.

2.4 Visualização

Durante a busca, cada algoritmo pode gravar dois tipos de evento: a entrada de um vértice na fronteira e a sua expansão. A visualização reproduz essa lista, sem interferir na lógica dos algoritmos, e gera para cada algoritmo uma animação (GIF e MP4) com um quadro por vértice expandido. Os vértices aparecem em cinza (não vistos), amarelo (na fronteira) e azul (expandidos), o vértice expandido no passo atual aparece em vermelho e maior, e no fim o caminho é desenhado em verde. O título acompanha o passo e o número de nós expandidos e, no fim, mostra a distância final. Quando o Hill Climbing falha, a sequência de vértices expandidos forma um caminho no grafo, e ela é desenhada como uma linha vermelha tracejada até o mínimo local. Durante as medições de tempo, a gravação de eventos é desligada.

3 Metodologia experimental

Todas as redes e todos os sorteios usam semente fixa (42), o que torna os resultados reproduzíveis.

Experimento i (visualização). Foi gerada uma rede com $n = 300$ e $k = 5$. Foram sorteados pares com distância em linha reta de pelo menos 200 entre origem e destino, para que a busca atravesse boa parte da rede, e foram escolhidos dois: um em que todos os algoritmos chegam ao destino e outro em que o Hill Climbing fica preso em um mínimo local.

Experimento ii (estatísticas). Foram geradas redes com $n = 2500$ e $k = 3$, 7 e 11, e sorteados 30 pares distintos, acima do mínimo de 10 pedido. As três redes usam a mesma semente, então os vértices ficam nas mesmas posições. Como os k vizinhos mais próximos de um vértice estão sempre entre os k' mais próximos para $k' > k$, toda aresta da rede com $k = 3$ existe também nas redes com $k = 7$ e $k = 11$. Assim, os pares foram sorteados uma única vez, na maior componente da rede com $k = 3$, e usados nas três redes, de modo que a comparação entre valores de k é feita sobre exatamente os mesmos pares. Cada busca foi executada 20 vezes com o coletor de lixo do Python desligado, e o tempo do par é a média dessas execuções. A distância média e o seu desvio padrão consideram apenas

as execuções que chegaram ao destino, e os desvios são amostrais. Os tempos foram medidos em um processador Intel Core i5-1335U (13ª geração), com 8 GB de RAM, no sistema operacional Pop!_OS 22.04 LTS, com Python 3.10.12.

Experimento iii (A* vs Dijkstra). Sobre as mesmas execuções do experimento ii, foram comparados, par a par, a distância encontrada, os nós expandidos e o tempo, além da relação entre os nós expandidos e a distância em linha reta entre origem e destino. Nos mesmos pares e redes, também foi executado o A* com a heurística de Chebyshev, com o tempo medido da mesma forma.

4 Resultados

4.1 Experimento i: visualização

A Figura 1 mostra o estado final dos seis algoritmos no par em que todos chegam ao destino, e a Tabela 2 traz a distância e os nós expandidos de cada um. Os caminhos completos (a lista de vértices de cada solução) são impressos pelo script do experimento e salvos em CSV. Por exemplo, o caminho ótimo encontrado pelo A* e pelo Dijkstra tem 14 vértices. As animações passo a passo acompanham este relatório e são apresentadas nos slides.

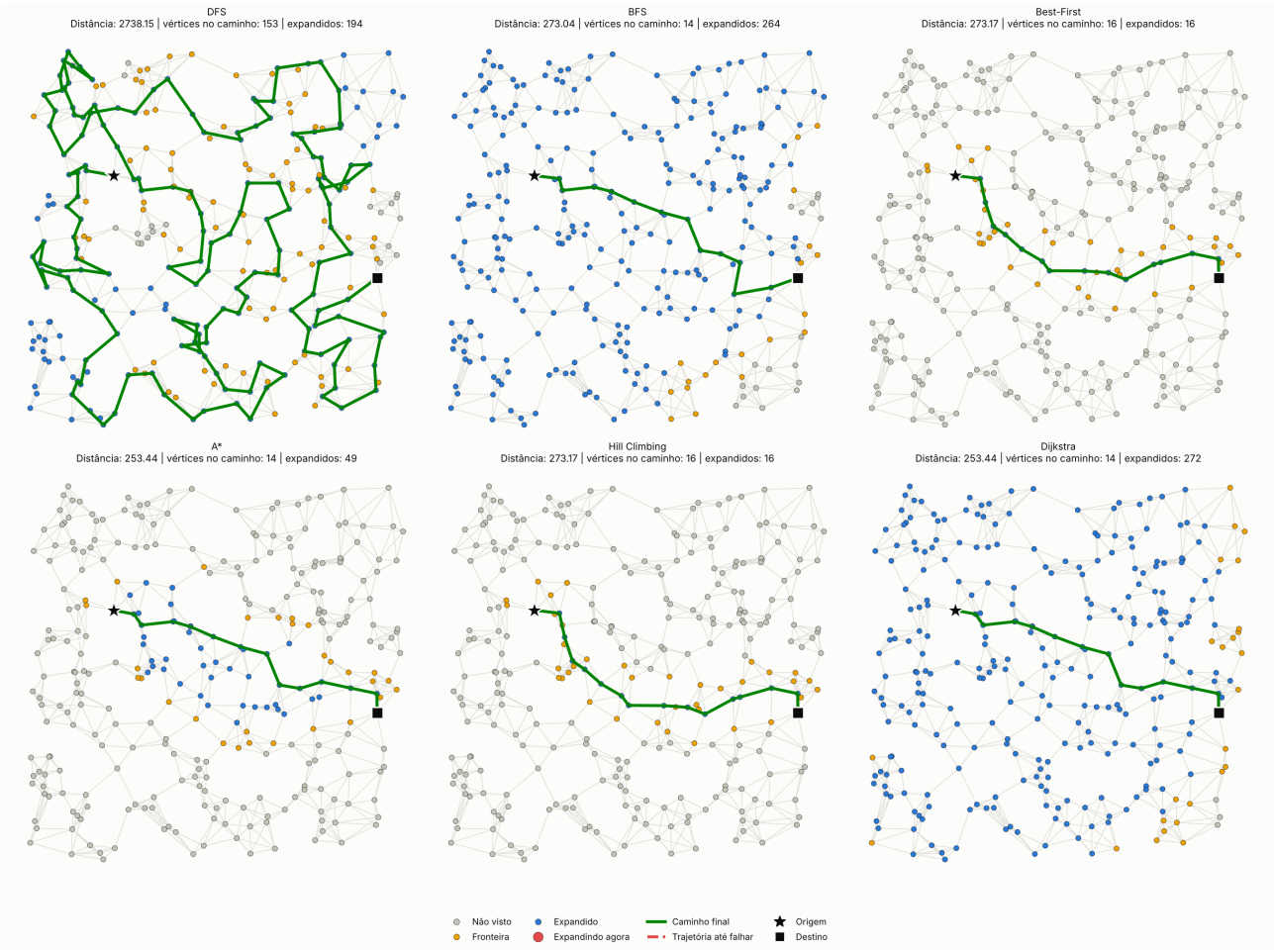


Figura 1: Estado final dos seis algoritmos na rede com 300 vértices e $k = 5$ (origem: estrela; destino: quadrado). Azul: vértices expandidos; amarelo: fronteira no momento em que a busca terminou; verde: caminho encontrado.

Tabela 2: Resultados do experimento i no par em que todos os algoritmos chegam ao destino.

Algoritmo	Distância	Vértices no caminho	Nós expandidos
DFS	2738,15	153	194
BFS	273,04	14	264
Best-First	273,17	16	16
A*	**253,44**	14	49
Hill Climbing	273,17	16	16
Dijkstra	**253,44**	14	272

No segundo par, o Hill Climbing para em um mínimo local depois de 12 expansões, enquanto o A* encontra o caminho ótimo, de distância 296,19, expandindo 84 vértices (Figura 2). Os outros quatro algoritmos também chegam ao destino nesse par, com comportamento qualitativamente igual ao da Figura 1.

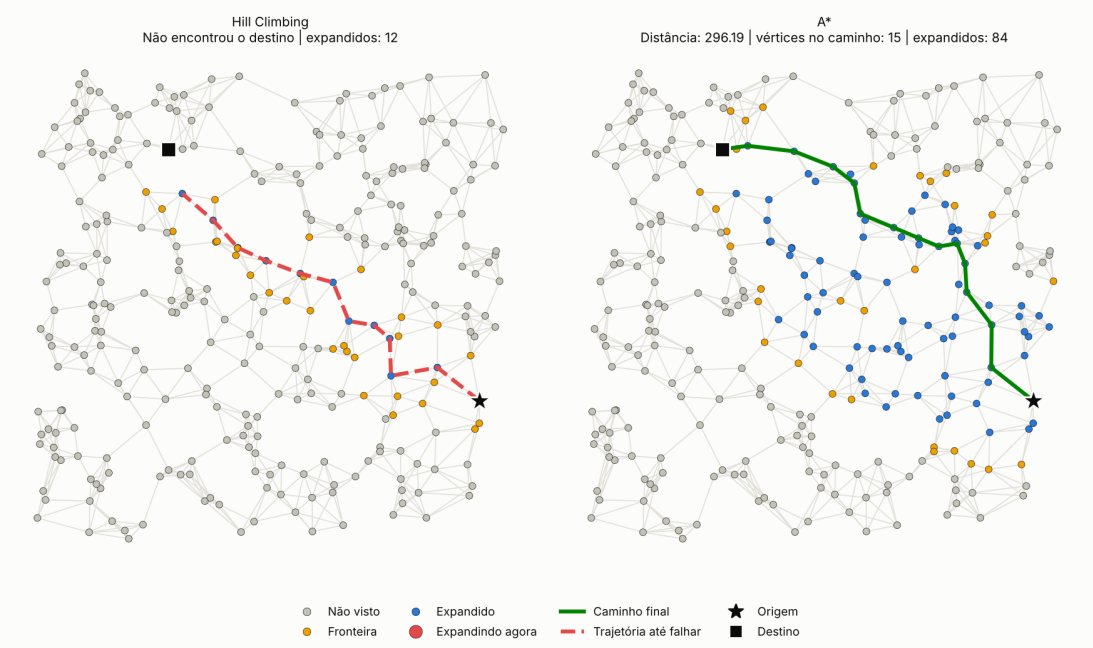


Figura 2: Par em que o Hill Climbing fica preso em um mínimo local (esquerda, trajetória tracejada), comparado ao A* no mesmo par (direita).

4.2 Experimento ii: estatísticas em redes grandes

A Tabela 3 apresenta as médias sobre os 30 pares para cada configuração.

Tabela 3: Resultados do experimento ii ($n = 2500$, 30 pares). Distância e desvio consideram apenas as execuções que chegaram ao destino; para o Hill Climbing, a coluna Sucessos indica sobre quantos pares essas médias foram calculadas.

k	Algoritmo	Dist. média	Desvio	Tempo (ms)	Desvio (ms)	Expandidos	Sucessos
3	DFS	13643,71	7825,91	0,64	0,39	1125,1	30
	BFS	2043,15	1027,07	0,43	0,25	1289,9	30
	Best-First	2298,82	1176,92	0,10	0,06	72,7	30
	A*	1954,92	973,03	0,62	0,54	422,1	30
	Hill Climbing	338,94	395,76	0,01	0,01	5,5	2
	Dijkstra	1954,92	973,03	1,22	0,71	1289,8	30
7	DFS	55936,54	21479,69	1,21	0,58	1223,7	30
	BFS	1627,36	845,52	0,66	0,38	1299,4	30
	Best-First	1633,10	862,26	0,08	0,04	24,5	30
	A*	1478,86	764,99	0,48	0,43	195,7	30
	Hill Climbing	1485,87	896,94	0,06	0,03	20,8	23
	Dijkstra	1478,86	764,99	1,91	1,13	1285,9	30
11	DFS	71923,82	41789,98	1,43	0,90	1091,7	30
	BFS	1535,66	789,45	0,91	0,51	1321,2	30
	Best-First	1508,07	763,37	0,09	0,05	18,3	30
	A*	1422,54	734,06	0,42	0,34	125,3	30
	Hill Climbing	1508,07	763,37	0,07	0,03	18,3	30
	Dijkstra	1422,54	734,06	2,52	1,47	1286,9	30

4.3 Experimento iii: A* vs Dijkstra

Nos 90 pares (30 para cada k), o Dijkstra e o A* com as duas heurísticas encontraram exatamente a mesma distância, como previsto pela admissibilidade. A Tabela 4 resume as diferenças de esforço, e a Figura 3 mostra os nós expandidos por cada algoritmo em função da distância em linha reta entre origem e destino.

Tabela 4: Comparação entre o A* com as heurísticas euclidiana (Eucl.) e de Chebyshev (Cheb.) e o Dijkstra ($n = 2500$, 30 pares por k). A redução de nós expandidos é em relação ao Dijkstra, e a razão de tempo indica quantas vezes o Dijkstra é mais lento que cada A* (média das razões calculadas par a par).

	Nós expandidos			Redução		Razão de tempo	
k	A* Eucl.	A* Cheb.	Dijkstra	Eucl.	Cheb.	Eucl.	Cheb.
3	422,1	510,9	1289,8	67,3%	60,4%	2,6×	2,5×
7	195,7	347,3	1285,9	84,8%	73,0%	5,4×	3,3×
11	125,3	313,1	1286,9	90,3%	75,7%	7,2×	3,5×

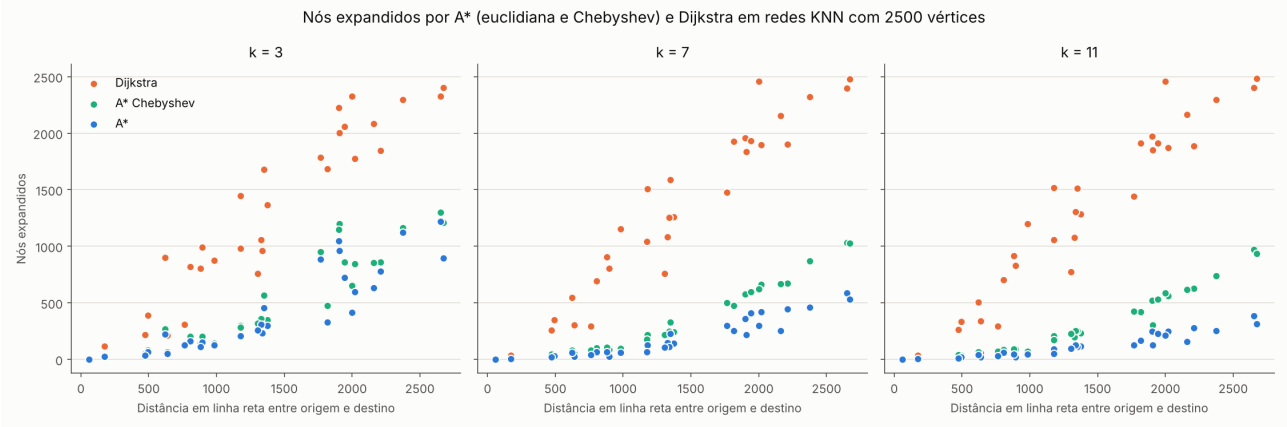


Figura 3: Nós expandidos pelo A* com as heurísticas euclidiana e de Chebyshev e pelo Dijkstra em função da distância em linha reta entre origem e destino, para cada valor de k.

5 Discussão

Qualidade das soluções. A* e Dijkstra encontram sempre o caminho de menor distância. Entre os algoritmos sem garantia de otimalidade, a BFS fica entre 5% e 10% acima do ótimo nas médias da Tabela 3, porque minimiza o número de arestas e ignora os pesos, mas em uma rede KNN as arestas ligam apenas vértices próximos e têm comprimentos da mesma ordem, e por isso o caminho com menos arestas costuma ser razoável. O Best-First fica 18% acima do ótimo com $k = 3$ e apenas 6% com $k = 11$: com poucas arestas, seguir sempre na direção do destino frequentemente leva a desvios, enquanto com muitas arestas quase sempre existe um vizinho na direção certa. A DFS é de longe a pior, com distâncias de 7 a 50 vezes maiores que a ótima, e piora com k: quanto mais arestas, mais caminhos existem para a pilha percorrer antes de topar com o destino, o que aparece na Figura 1 como um caminho que serpenteia por quase toda a rede.

Esforço e tempo. A BFS e o Dijkstra expandem quase toda a componente (cerca de 1290 dos 2500 vértices, em média) independentemente de k, porque crescem em todas as direções até alcançar o destino. Na Figura 1, isso aparece como a rede quase inteira pintada de azul. O A* explora uma faixa em torno da linha reta entre origem e destino, e o Best-First vai quase direto ao destino, expandindo praticamente só os vértices do caminho. Os tempos acompanham o número de nós expandidos, mas não na mesma proporção, porque cada expansão do A* é mais cara que uma do Dijkstra: além das operações no heap, que os dois fazem, o A* calcula a heurística para cada vizinho. Com $k = 11$, cada expansão do A* custa cerca de 70% a mais, mas ele expande 10 vezes menos vértices, e por isso ainda é cerca de 7 vezes mais rápido. Como as buscas levam poucos milissegundos, os tempos absolutos dependem da máquina e da implementação em Python, e as conclusões mais robustas vêm da contagem de nós expandidos.

Hill Climbing e mínimos locais. Sem fronteira e sem backtracking, o Hill Climbing é o algoritmo mais barato, mas a sua taxa de sucesso depende fortemente de k: 2 de 30 pares com $k = 3$, 23 com $k = 7$ e 30 com $k = 11$. Um mínimo local ocorre quando nenhum vizinho do vértice atual está mais perto do destino que ele, o que é comum com poucos vizinhos por vértice e com as lacunas que a distribuição aleatória dos pontos deixa no plano, como mostra a Figura 2. Quando nenhum mínimo local aparece no caminho, o Hill Climbing segue exatamente o mesmo caminho do Best-First (Tabela 2 e linhas de $k = 11$ da Tabela 3), porque os dois escolhem sempre o vértice de menor h; a diferença é que o Best-First guarda os outros vértices na fronteira e consegue voltar atrás quando trava. A distância média do Hill Climbing com $k = 3$ parece baixa apenas porque os dois pares em que ele teve sucesso eram pares próximos, e não deve ser comparada com as dos outros algoritmos.

A* vs Dijkstra. A Figura 3 mostra que, nos dois algoritmos, os nós expandidos crescem com a distância entre origem e destino, mas no Dijkstra crescem muito mais rápido: ele expande todos os vértices a uma distância da origem menor que a do destino, enquanto o A* se concentra nos que parecem estar no caminho. Por isso a vantagem do A* é maior justamente nos pares distantes. Além disso, o esforço do Dijkstra praticamente não muda com k, enquanto o do A* cai de 422 para 125 nós

expandidos. Com mais arestas, os caminhos ótimos ficam mais próximos da linha reta, a heurística passa a estimar o custo real com mais precisão e o A* precisa explorar menos alternativas.

Efeito da heurística. O A* com a heurística de Chebyshev fica sempre entre o A* euclidiano e o Dijkstra (Tabela 4 e Figura 3). Isso ilustra que, entre heurísticas admissíveis, a que estima o custo real com mais precisão leva a menos expansões; no extremo $h = 0$, o A* se torna o próprio Dijkstra. A diferença entre as duas heurísticas cresce com k: com $k = 3$, o A* com Chebyshev expande 21% mais vértices que o euclidiano, e com $k = 11$, 2,5 vezes mais. Com mais arestas, os caminhos ótimos se aproximam da linha reta e a heurística euclidiana fica quase exata, enquanto a de Chebyshev continua subestimando nos destinos em diagonal. No tempo, o A* com Chebyshev chega a ficar mais lento conforme k cresce, porque o número de expansões cai pouco enquanto cada expansão passa a examinar mais vizinhos.

Sobre as visualizações. As animações tornam visível a diferença entre os critérios de escolha da fronteira, que nas tabelas aparece apenas como números. A BFS e o Dijkstra crescem como manchas aproximadamente circulares em torno da origem, o A* forma uma faixa alongada em direção ao destino, o Best-First avança quase em linha reta e a DFS se afasta e volta pela rede de forma aparentemente desordenada. No caso do Hill Climbing, a trajetória tracejada mostra exatamente o vértice em que a busca travou e por que: a partir dali, todos os vizinhos se afastam do destino. Usar o mesmo par e a mesma rede para todos os algoritmos foi essencial para que essas comparações visuais fossem diretas.

6 Conclusão

Os seis algoritmos foram implementados e avaliados em redes KNN, e os resultados confirmam o comportamento esperado de cada um. A* e Dijkstra encontram sempre o caminho ótimo, mas o A* expande até 90% menos vértices, e a sua vantagem cresce com k e com a distância entre origem e destino. O Best-First oferece soluções próximas da ótima a um custo muito baixo, especialmente em redes mais densas, enquanto a DFS produz caminhos muito longos. O Hill Climbing sem backtracking é eficiente quando funciona, mas fica preso em mínimos locais com frequência em redes esparsas. A gravação de eventos durante a busca permitiu visualizar todos os algoritmos com o mesmo código, sem alterar sua lógica, e as animações deixaram evidente como cada critério de escolha molda a exploração do espaço de busca. A comparação com a heurística de Chebyshev mostrou que, mesmo mantendo a otimalidade, uma heurística menos precisa reduz bastante a vantagem do A* sobre o Dijkstra. Como trabalhos futuros, seria interessante testar heurísticas não admissíveis, como o A* ponderado, que troca a garantia de otimalidade por menos expansões, e repetir os experimentos em outros tipos de rede, como a rede geográfica, em que as arestas longas são mais raras.

Referências

[1] S. Russell e P. Norvig. *Artificial Intelligence: A Modern Approach*. 4. ed. Pearson, 2021.

[2] P. E. Hart, N. J. Nilsson e B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, v. 4, n. 2, p. 100–107, 1968.

[3] E. W. Dijkstra. A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, v. 1, p. 269–271, 1959.